Hostel Complaint & Attender Duty Tracking System

Phase 1 — Design Document

Ajoy Mathew • CS23B101 • CS5013 Programming with AI

Stakeholder: Hostel Manager, Cauvery Hostel, IIT Madras • 11 September 2026

1 Problem & Scope

The Hostel Manager at Cauvery Hostel processes daily maintenance complaints entirely by hand. Residents submit a Google Form; the manager copies each response into a spreadsheet, sorts it into a domain (Electrical, Civil, RO Plant), checks which of four attenders covers that floor and is on duty, assigns the work verbally, chases inspection dockets, and at month end tallies points for bonus calculation. This costs **30–45 minutes every day** and is error-prone at every step.

Points account at **+2** per resolved complaint, custom values for special duties, and **−1** per logged infraction — currently reconciled in a spreadsheet that is easily corrupted when edited from mobile devices.

1.1 In scope

S1. Automated ingestion from the Google Form response sheet, with a CSV upload fallback, and no duplicate complaints ever created.
S2. Categorization into Electrical / Civil / RO Plant / Network, with a confidence gate that routes uncertain text to a manager triage queue rather than guessing.
S3. Heuristic dispatch to an attender, scoring floor coverage, live availability and current workload.
S4. Field verification: the attender records the IITM EU CMS docket number, which is a precondition for earning points.
S5. Resolution workflow with a formal state machine, and **points** on an append-only ledger (+2 resolutions, custom special duties, –1 penalties).
S6. Push notifications to attender phones when work is assigned or escalated.
S7. Manager dashboard: live counts, triage, leaderboard, MTTR, bottlenecks, anomaly alerts, and one-click `.xlsx` export.
S8. Audit: every state change and point movement reconstructible.

1.2 Out of scope

- **Resident accounts.** Residents keep using the existing Google Form; they never log in.
- **Replacing or integrating with the IITM EU CMS app.** No API is available to us. We store its docket number for traceability but can never verify it upstream.
- **A native mobile app.** Attenders use an installable web app (PWA); push arrives through it, not a Play Store binary.
- **Multi-hostel tenancy.** Single-hostel deployment; the schema models no hostel dimension.
- **Payroll.** We compute points; money is handled outside the system.

1.3 Actors

Actor	Count	Device	Responsibilities
Hostel Manager	1	Desktop	Oversight, triage, reassignment, penalties, special duties, exports, account provisioning
Attender	∼5	Android/IOS phone	Receive push alerts, work assigned queue, record CMS docket, resolve, claim duties, view own standing
Resident	hundreds	any	Submits the Google Form. *Never authenticates.*
Scheduler	—	—	Polls the sheet, ages priorities, runs nightly telemetry scans

1.4 Scale envelope

≤50 complaints/day, ≤10 concurrent users, ∼50k rows/year. Every sizing decision in this document follows from these numbers — and several designs were rejected precisely because they solved a scale problem this

system will never have.

2 Authentication & Access Control

One mechanism for everyone: Google sign-in, admitted by an allowlist.. There is no self-registration: every principal is a pre-provisioned `app_user` row created by a manager.

	Every user — manager and attenders alike
Mechanism	Google OIDC (`openid, email, profile`)
Credential stored	**None**
Admission decided by	Verified email present and `active` in `app_user`
Role	`app_user.role`
Re-authentication	One tap; the account is already signed in on the device

The allowlist is the access-control boundary. Google establishes *who someone is*; a custom `OidcUserService` resolves the verified email against `app_user` and throws if there is no matching active row. Emails are normalised before comparison and matched only against the `email_verified` claim

2.1 Authorization matrix

Role *and* ownership are both checked. Role alone is never sufficient for a task action, otherwise any attender could resolve another's work and collect the points.

Capability	**Manager**	**Attender**
View all complaints; trigger ingestion; upload CSV	yes	no
Classify, assign, reassign, reject, reopen	yes	no
Start / inspect / record docket / resolve	via `act-as`	**own assignments only**
Claim special duties	no	yes
Log penalties, adjustments, post special duties	yes	no
View leaderboard	full	ranks + own detail
Provision users, edit the allowlist, deactivate accounts	yes	no
Export reports; edit lexicon, weights, SLA table	yes	no

`act-as` lets the manager perform an attender's transition during roll-call when that person cannot use their phone. It runs the identical state machine but audits `actor = manager, on_behalf_of = attender`; the points still go to the attender.

3 System Architecture

3.1 A modular monolith, deployed as one Spring Boot jar

One executable `.jar` (Spring Boot 3.x, embedded Tomcat), organised internally into bounded modules with explicit interfaces. This is a deliberate choice over microservices:

- At $\leq$50 complaints/day with 4-8 users, distribution would add network failure modes, deployment complexity and eventual consistency in exchange for scale we will never need.
- Module boundaries are drawn so a component *could* be extracted later, and are enforced in CI by ArchUnit.

Within modules we use **ports and adapters** only where an external system is involved. The ingestion port is what lets the Google Sheets adapter and the CSV fallback adapter share one normalization, dedupe and categorization pipeline instead of drifting into two code paths.

3.2 Context and processing pipeline

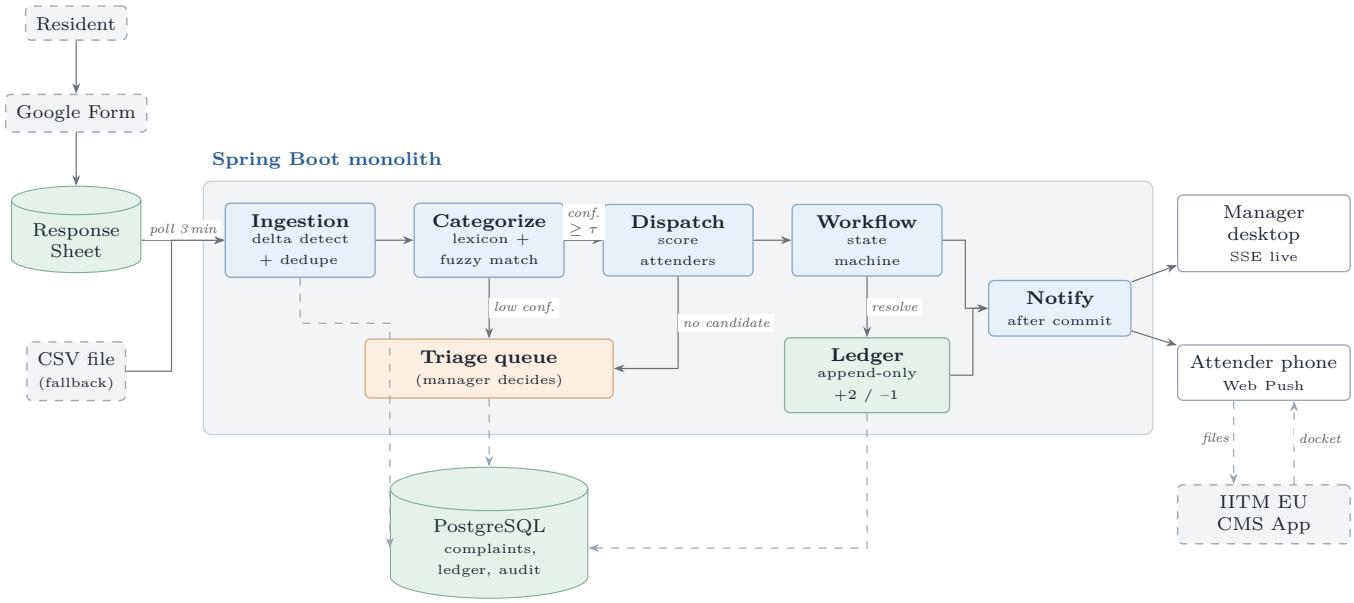


Figure 1: Context and processing pipeline. Dashed edges are external or persistence links; the IITM EU CMS app has *no* programmatic integration — the attender files there manually and types the returned docket number back into our system.

3.3 Module breakdown

Each module owns exactly one invariant, which is what makes the boundary meaningful.

Module	Java package	Purpose & the invariant it owns
ingestion	`...ingestion`	Fetch rows, detect deltas, normalize into complaints. *A source row never becomes two complaints.*
categorization	`...categorization`	Assign domain + confidence to complaint text. *Low confidence never silently becomes a wrong domain.*
dispatch	`...dispatch`	Score attenders, produce assignments. *Every complaint gets an assignment or a recorded triage reason.*
workflow	`...workflow`	State machine, guards, transitions. *Only legal transitions occur, atomically with their side effects.*
ledger	`...ledger`	Point movements, leaderboard projection. *Append-only; no double-credit; balance is derived, never stored.*
notification	`...notification`	Outbox fan-out to manager SSE and attender Web Push. *Best-effort; never blocks a transaction.*
analytics	`...analytics`	MTTR, bottlenecks, anomaly detection. *Read-only over ledger and audit.*
reporting	`...reporting`	`.xlsx` generation via Apache POI. *Reports are reproducible point-in-time snapshots.*
identity	`...identity`	Users, roles, Google allowlist, provisioning. *No self-registration*
audit	`...audit`	Append-only event log. *Written in the same transaction as the change it records.*

3.4 Layering

The application is divided into four layers, each with a clear responsibility:

[`web` → `application` → `domain`]

with `infrastructure` providing the implementations needed to talk to databases and external services.

- **`web` handles requests and responses.** Controllers authenticate the user, validate the request and call an application service. They never access repositories directly.
- **`application` implements use cases.** It coordinates a complete operation, such as resolving a complaint or assigning a task. This is also where transaction boundaries are defined.

- `domain` **contains the business rules.** It decides what the system allows, such as requiring a CMS docket before a complaint can be resolved. The domain layer depends only on the JDK and has no knowledge of Spring, JPA or HTTP.
- `infrastructure` **handles external systems.** Database repositories, Google Sheets ingestion and push notifications live here. They implement interfaces (ports) defined by the application or domain rather than exposing their internal details.

The boundaries are enforced by ArchUnit in CI. In particular, controllers cannot bypass the application layer, modules communicate only through their published interfaces, and `@Transactional` is used only at application-layer use-case boundaries. This keeps business logic independent of infrastructure and makes each module easier to test and change.

3.5 Attender Interface

The attender dashboard provides:

- **Assigned tasks:** Complaint, category, priority, room number, floor, SLA status, and assignment time.
- **Student details:** Name, roll number, and phone number for assigned complaints.
- **Task actions:** `START_INSPECTION` to begin inspection, `CONFIRM_DEFECT` after identifying a defect, `RECORD_DOCKET` to enter the EU CMS docket number, `MARK_RESOLVED` after completing the repair, `ON_HOLD` when work is blocked, and `RESUME` when the blocker is removed.
- **Resolution details:** Add notes and record the EU CMS docket number when required.
- **Points:** View points earned from completed duties.

3.6 Manager Interface

The manager dashboard provides:

- **Complaint overview:** View all complaints, their category, priority, status, SLA status, assignment, and history.
- **Complaint actions:** Manually assign or reassign complaints, change priority, release an assignment back to the pool, reject complaints, and reopen resolved complaints.
- **Triage:** Review unclassified complaints or complaints for which no suitable attender was found, and either assign or reject them.
- **Attender management:** Add or deactivate attenders, configure floor coverage, and view current workload and points.
- **Student and complaint details:** View student name, roll number, phone number, contract number, room number, complaint details, and classification information.
- **Configuration:** Configure priority rules, SLA windows, working hours, escalation thresholds, assignment weights, and load limits.
- **Synchronization:** Trigger `Sync Now` and view the latest synchronization status and ingestion errors.
- **Audit and history:** View assignment changes, status transitions, points ledger entries, and other audit information.

3.7 Deployment topology

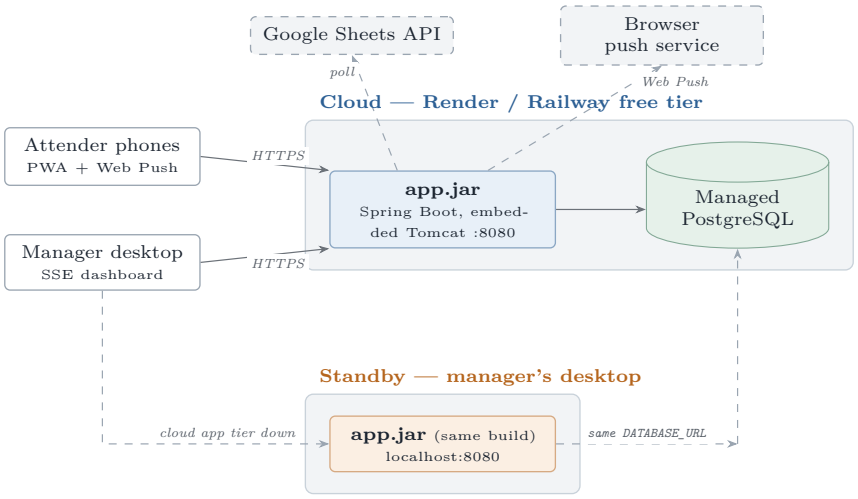


Figure 2: Deployment. **One database.** The standby is the same jar with the same `DATABASE_URL`, started on the manager's desktop when the cloud app tier is down. The outage designed for is the *cloud platform* failing

	Cloud (primary)	Standby (manager's desktop)
Database	Managed PostgreSQL	**The same** managed PostgreSQL
Reached by	Attenders and manager, HTTPS	Manager only, on `localhost`
Ingestion	Sheets polling + CSV	Both — internet is available
Notifications	SSE + Web Push	SSE + Web Push
Authentication	Google (attenders) + password (manager)	Manager password — no Google round-trip
Purpose	Day-to-day operation	Continuity when the app tier is down

Free-tier instances sleep when idle, so `@Scheduled` polling alone is unreliable: while the application is asleep, there is no running scheduler to poll the response sheet. To make ingestion event-driven, a Google Apps Script trigger attached to the response sheet sends an HTTP request to the application whenever a new form response arrives; the hosting platform can then wake the application to process the request. The endpoint only signals that the sheet has changed — the application still performs its normal watermark-based delta detection and deduplication, so repeated triggers are harmless. A **Sync Now** button gives the manager an explicit manual wake-and-sync path if the trigger fails or immediate reconciliation is needed.

4 Data Model

4.1 Design principles

1. **Keep historical facts unchanged.** Point movements are recorded in an append-only ledger rather than by modifying a running points balance. The current balance is calculated as the sum of these entries, making every credit or penalty independently traceable.
2. **Protect concurrent updates.** Mutable records such as complaints and assignments carry a `version` number. Optimistic locking uses this number to detect when two people try to modify the same record at the same time, preventing one change from silently overwriting another.
3. **Record changes together with the changes themselves.** Every audit entry is written in the same database transaction as the operation it describes. If the operation fails, its audit entry is rolled back too; if it succeeds, the corresponding audit entry is saved.
4. **Keep important records instead of deleting them permanently.** Operational records are not hard-deleted. When something should no longer appear in normal views, it is marked as deleted or inactive while the original record remains available for audit and historical analysis.

4.2 Entity–relationship model

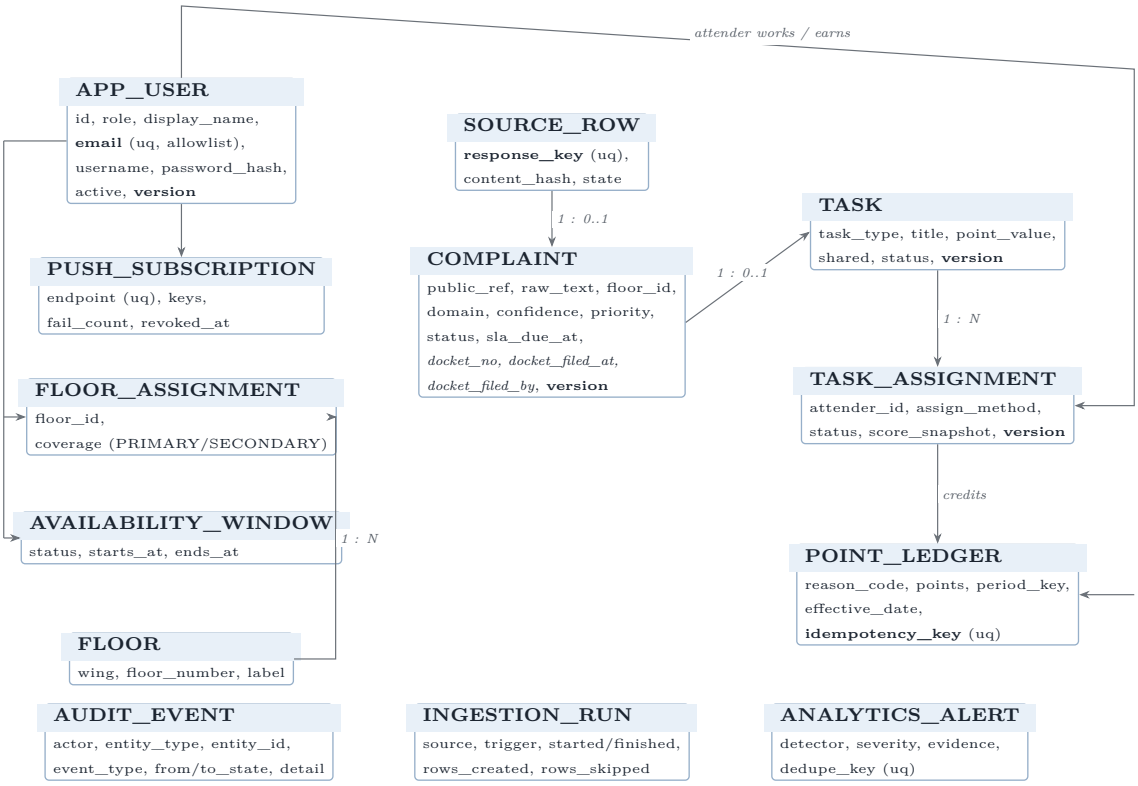


Figure 3: Core entities. Bold fields carry the guarantees: `version` for optimistic locking, `response_key` for ingestion identity, `idempotency_key` for point integrity.

4.3 The two constraints that carry the design

Constraint	What it prevents
`uq_source_row` (source, **response_key**)	A source row becoming two complaints. Identity is `SHA256(timestamp ‖ email)` — *not* row index, because deleting one sheet row shifts every index below it and a positional key would report the whole tail as amended.
`uq_ledger_idem` (**idempotency_key**)	Double-crediting. The key is derived from the *work* (`RESOLVE:{task}:{attender}`), not the request, so a retry on flaky hostel Wi-Fi — the expected case, not an exotic one — collides and is swallowed as a no-op.

Consequence. There is no `attender.points` column anywhere in the schema. No counter to increment means no lost update on a counter, and nothing to reconcile. A balance is a `SUM` over immutable facts, which is also what lets a bonus dispute be answered line by line.

4.4 Audit log

Every state change, point movement, authentication attempt and configuration change is written to `audit_event`. It is the evidence base for bonus disputes, so it is designed to be trustworthy rather than decorative.

Column	Content
`occurred_at`	UTC instant of the change
`actor_id`, `actor_role`	Who did it. `NULL` for system/scheduler actions
`on_behalf_of`	Set when the manager used `act-as`; the points still go to the attender
`entity_type`, `entity_id`	What was changed — `COMPLAINT`, `TASK`, `LEDGER`, `APP_USER`, ...
`event_type`	`TRANSITION`, `POINTS_CREDITED`, `PENALTY_LOGGED`, `SIGN_IN`, `SIGN_IN_REJECTED`, `USER_PROVISIONED`, `CONFIG_CHANGED`, ...
`from_state`, `to_state`	For transitions, the exact edge taken
`detail`	JSON payload: reason text, dispatch score snapshot, old/new config values
`correlation_id`	Groups everything caused by one request or one ingestion run

Two properties make it dependable. It is written by a `BEFORE_COMMIT` transactional event listener, so it **commits atomically** with the change it records and can never silently diverge. And there is **no code path that updates or deletes an audit row**. Combined with the append-only ledger, an attender's monthly bonus can be reconstructed from first principles and defended line by line.

5 Complaint & Task Lifecycle

Status is the most-read field in the system: it drives the attender queue, the dashboard, SLA aging, MTTR measurement and the point credit.

The *only* way to change it is `WorkflowService.transition(id, event, actor, payload)`, which loads the aggregate with its `version`, rejects any `(state, event)` pair absent from the transition table, evaluates the guard, then applies the change, its side effects and the audit row **in one transaction**; notifications fire separately on `AFTER_COMMIT`.

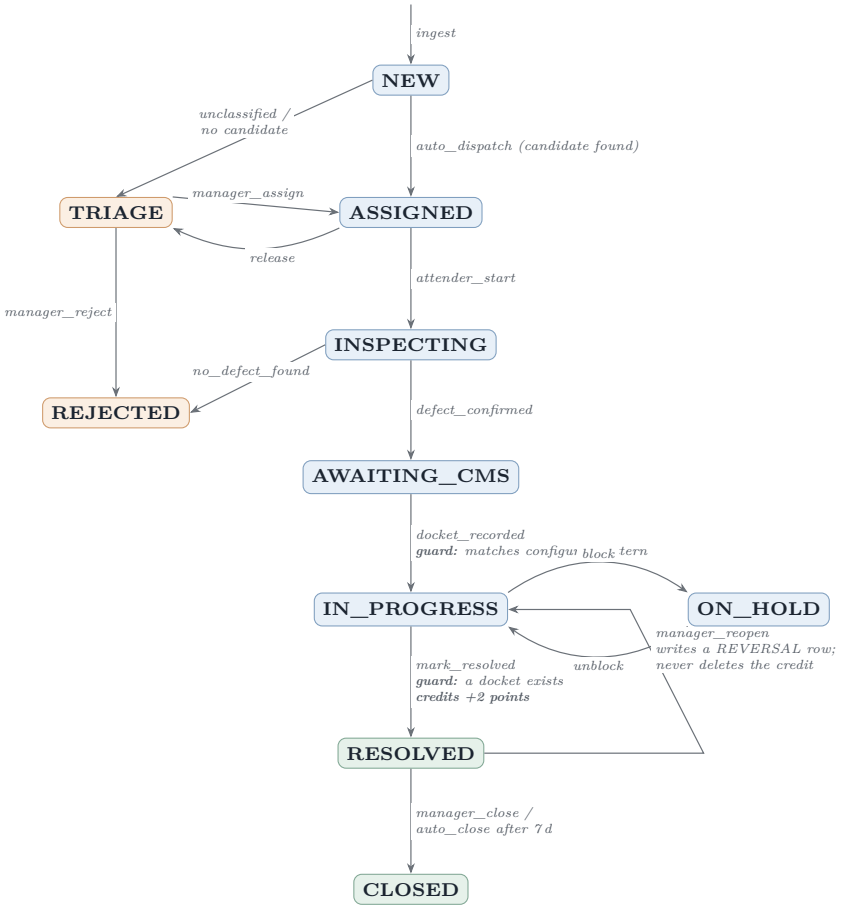


Figure 4: Complaint lifecycle. Two guards carry the institutional requirements: a docket must match the configured pattern to be recorded, and **a complaint cannot be resolved for points without one**.

5.1 Two guards worth calling out

The docket gate. `mark_resolved` requires a docket to exist. This makes institutional traceability structurally enforced rather than a matter of attender diligence — you cannot collect +2 points for a complaint you never filed with the Engineering Unit.

Reopen reverses, never deletes. The ledger is append-only, so a reopen writes a compensating `REVERSAL` row; both the original credit and the reversal stay visible. Re-resolving afterwards writes a new credit under key `RESOLVE:{task}:{attender}:r{n}`, so the idempotency guarantee survives the reopen cycle without ever blocking legitimate repeat work.

5.2 Special duties

Special duties have no complaint and no docket: `OPEN` → `CLAIMED` → `COMPLETED`, with `CANCELLED` and `EXPIRED` exits. For a **shared** duty several attenders hold active assignments at once; the duty completes when every assignee has marked their part done, and *each* is credited once under their own key `DUTY:{task}:{attender}`.

5.3 Priority logic

Priority is not a fixed label — it ages, so old low-priority work eventually surfaces above new noise. The SLA timings below are measured in **working hours**, not wall-clock hours. This prevents a complaint submitted overnight from consuming its SLA while no attender is expected to be working. The working calendar (working hours, weekends and holidays, and initial SLA) is configurable by the manager.

Base	Assigned when	Initial SLA
`P1`	Safety keywords matched **exactly** — *shock, spark, short circuit, sewage, no water, overflow.* .	Electrical 4 hOther 8 h
`P2`	Default for a confidently classified complaint	Elec 8 h, RO 12 h,Civil / Network 24 h
`P3`	Downgraded by the manager	72 h

A scheduled job escalates `P3` → `P2` → `P1` once a complaint has been open past **1.5**× its SLA window without reaching `RESOLVED`. `base_priority` is retained, so the dashboard can always show "P1 (aged from P3)". Escalation caps at `P1`, and **hold time does not count toward it** — otherwise "waiting three days for the Engineering Unit to send a plumber" would be recorded as attender slowness and would point the bottleneck analysis at exactly the wrong place.

6 Dispatch Heuristic

Given a classified complaint c and attenders A, choose who should do the work. The formulation must always terminate with an assignment *or* a recorded triage reason, be deterministic including tie-breaks, be explainable to the manager, avoid starvation, and be tunable without a redeploy.

6.1 Stage 1 — hard eligibility filter

$$E = \{\, a \in A \;:\; a.\text{active} \;\wedge\; \text{availability}(a, \text{now}) = \texttt{AVAILABLE} \;\wedge\; \neg\,\text{assigned}(a, c) \,\}$$

Availability is a **gate, not a score**. Someone on leave must never be chosen no matter how well they score elsewhere; folding it into a weighted sum would let a strong floor match outvote the fact that a person is not at work. If $E = \varnothing$ the complaint goes to `TRIAGE` with reason `NO_AVAILABLE_ATTENDER`.

6.2 Stage 2 — score the eligible set

$$\text{Score}(a, c) = w_f \cdot \text{Floor}(a, c) \;+\; w_l \cdot \text{Load}(a)$$

The score uses two factors: how well the attender covers the complaint's floor and how much free capacity they currently have. Both terms are normalised to $[0, 1]$, and the weights sum to 1, so the final score also lies in $[0, 1]$. The weights are configurable by the manager.

Term	Weight	Definition
Floor proximity	$w_f = 0.60$	`PRIMARY` coverage = 1.00; `SECONDARY` coverage = 0.70; adjacent floor = 0.40; each additional floor of distance reduces the score by 0.05, with a minimum of 0.10. Non-zero scores for distant floors ensure that a complaint can still be assigned when its usual attender is unavailable.
Load (free capacity)	$w_l = 0.40$	$L(a) = \sum_t \pi(t)$ over active tasks with $\pi(\text{P1})=3, \pi(\text{P2})=2, \pi(\text{P3})=1$; $\text{Load}(a) = 1 - \min(1, L(a)/L_{cap})$, $L_{cap} = 8$. Weighting by priority reflects reality — three P1 electrical faults should count as a heavier workload than three P3 cosmetic complaints. `ON_HOLD` tasks count as 0.

6.3 Stage 3 — select, tie-break, record

best $= \arg\max_{a \in E} \text{Score}(a, c)$, with ties broken in strict order: (1) fewest tasks assigned *today* ; (2) longest time since last assignment; (3) lowest user id, which guarantees full determinism.

The complete score breakdown is persisted to `task_assignment.score_snapshot` as JSON (chosen attender, each component, the weights, runners-up, tie-break used).

6.4 Degenerate cases

Case	Outcome
No attender available ($E = \varnothing$)	`TRIAGE` / `NO_AVAILABLE_ATTENDER`
Room number unparseable	`TRIAGE` / `UNMAPPED_LOCATION` — dispatching on a location we could not read is worse than asking
Floor exists, nobody covers it	Assign least-loaded, warn `UNCOVERED_FLOOR`
Every attender at capacity	Assign best anyway, notify `CapacityExceeded`. Overflowing visibly beats triaging silently
Complaint `UNCLASSIFIED`	Never reaches dispatch; triaged at classification
Manager disagrees	`manager_reassign`; both assignments audited

7 PII & Data Protection

Complaints originate from residents, so the system handles personal information that is already present in the Google Form response sheet. The design follows a minimisation principle: the system stores only information needed for complaint handling, accountability and reporting, and access is restricted according to the user's role and the complaint's lifecycle.

Data	Why held	Retention / access
Student name	Allows the assigned attender to identify and contact the resident when handling a complaint; available to the manager for oversight	Attender: 7 days after resolution; Manager: retained with complaint record
Roll number	Identifies the resident and provides an unambiguous reference when handling the complaint; available to the manager	Attender: 7 days after resolution; Manager: retained with complaint record
Student phone number	Allows the assigned attender to contact the resident when clarification or access to the room is required	Attender: 7 days after resolution; Manager: retained with complaint record
Resident email	Contacting the reporter when details are unclear	90 days, then nulled
Room number	**Operationally essential** — dispatch depends on it and it identifies the location of the work	1 year
Complaint text	Diagnosis, classification and chronic-defect detection	1 year, then `[purged]`
Attender name	Assignment, accountability and leaderboard	Life of employment
Attender Google email	Allowlist key; identifies the actor in the audit log	Life of employment
Push endpoint + keys	Delivering assignment alerts	Until revoked or 3 failures
Audit log, point ledger	Bonus disputes, accountability and historical reconstruction	Indefinite

The system does not create additional resident profiles or accounts. MTTR, arrival counts and chronic-defect detection depend on timestamps, domain, room and other operational fields rather than the resident's identity. These analytics therefore continue to work after personal information is removed or hidden from an attender's view.

Push payloads. Push notifications contain the minimum information needed to alert the attender, such as the complaint reference and room number. They do not include the student's name, roll number or phone number, reducing the amount of personal information exposed through lock-screen previews or push relays.

During testing real student information is not used: names and phone numbers are replaced with synthetic values, roll numbers are replaced with synthetic identifiers, and room numbers are remapped through a fixed permutation that preserves floor structure so dispatch tests remain meaningful. The mapping table is not committed.

Secrets live only in environment variables — the Google service-account key (base64), `DATABASE_URL`, `APP_SECRET`, and the VAPID key pair. `.gitignore` blocks the obvious filenames and CI runs Gitleaks on every pull request, so a committed credential fails the build rather than reaching `main`. The Sheets service account is granted **read-only** access to the required response sheet.

8 Technology Stack by Module

Language Java 21 • **Build** Maven • **Framework** Spring Boot 3.x • **Database** PostgreSQL 16 • **Packaging** single executable `.jar`

Module	Libraries & technology	Milestone
identity	Spring Security 6: OAuth2 client (Google OIDC) for attenders, form login + BCrypt cost 12 for the manager, custom `OidcUserService` for the allowlist	S1 *(12–18 Sep)*
persistence	Spring Data JPA / Hibernate 6, Flyway (forward-only migrations), HikariCP, Testcontainers for tests	S1
ingestion	`google-api-services-sheets:v4` + service-account auth; Apache Commons CSV for the fallback adapter; SHA-256 via JDK `MessageDigest`	S2 *(19–25 Sep)*
categorization	Hand-built weighted lexicon; Apache Commons Text (Levenshtein, Jaro–Winkler) for fuzzy matching. *Apache OpenNLP `DocumentCategorizer` deferred to an optional second stage*	S2
dispatch	Plain Java domain code, no framework. Weights bound via `@ConfigurationProperties`	S3 *(26 Sep–2 Oct)*
workflow	Hand-rolled transition table over a Java `enum`. *Spring Statemachine was considered and rejected — our guards need transactional side effects, which is simpler to express directly*	S3
ledger	JPA with `@Version`; correctness carried by a database `UNIQUE` constraint rather than application logic	S3
scheduling	Spring `@Scheduled` / `@Async`; ShedLock for distributed lock safety	S2–S3
web (manager)	Thymeleaf, Tailwind CSS (purged), Chart.js 4	S4 *(3–9 Oct)*
web (attender)	Thymeleaf server-rendered shell + PWA service worker (push receiver)	S4
validation	Hibernate Validator (Jakarta Bean Validation) — entity constraints, CMS docket pattern	S3–S4
deployment	Docker Compose locally; `.jar` on Render / Railway with managed PostgreSQL	S4
concurrency	JPA optimistic locking; `SELECT ...FOR UPDATE` for duty claims only	S5 *(10–16 Oct)*
analytics	Plain Java statistics (median, MAD, EWMA, Poisson tail) over JPQL aggregates. *No stats library — the five estimators we need are a few lines each and stay explainable*	S6 *(17–23 Oct)*
reporting	Apache POI (`poi-ooxml`) for multi-tab `.xlsx` export	S7 *(24–30 Oct)*
notification	Spring MVC `SseEmitter` on `AFTER_COMMIT`; `nl.martijndwars:web-push` with VAPID for attender push	S7
testing	JUnit 5, AssertJ, Mockito, Testcontainers, ArchUnit, JaCoCo	Throughout
CI/CD	GitHub Actions (`mvn verify`), Gitleaks, OWASP Dependency-Check, Dependabot	S1

9 Test Plan

Level	Tooling	Scope	Share
Unit	JUnit 5, AssertJ	Pure domain logic — scoring, classification, transitions, statistics. No Spring context	70%
Slice	`@DataJpaTest`, `@WebMvcTest`	Repositories against real Postgres; controller contracts	15%
Integration	`@SpringBootTest` + Testcontainers	Full pipeline, transactions, concurrency	10%
Concurrency	`CountDownLatch`, `ExecutorService`	The five named races	named suite
Architecture	ArchUnit	Layering rules of Sec. 3.4	continuous
Acceptance	Stakeholder walk-through	The manager's own criteria	per milestone

All database tests run against real PostgreSQL via Testcontainers, never an in-memory substitute. The properties we most need to verify — unique-constraint behaviour under concurrent inserts, optimistic-lock failures, row locking — are precisely the ones an H2 stand-in would fake convincingly and wrongly. Coverage floor: 75% line, 90% on `domain`, gated in CI.

9.1 Fixtures

	Fixture	Purpose
F1	∼200 real complaints, anonymised	Classifier gold set; realistic dispatch
F2	∼60 hand-authored edge cases	Corner-case suite
F3	4 attenders × 25 tasks	Concurrency races
F4	90 days, ∼1800 complaints	Analytics and anomaly detectors
F5	∼30 malformed sheet rows	Ingestion robustness

9.2 Corner cases (selected from 31)

Case	Expected
Identical simultaneous submissions, two residents	Two complaints — distinct `response_key`
Byte-identical resubmission by one resident	One complaint — key collides
Same CSV uploaded twice by mistake	Zero duplicates
Sheet row deleted, shifting all indices below	**Zero spurious amendments**
Sheet row deleted entirely	Ignored — the sheet is an inbox, the complaint is unaffected
Required column renamed	Run fails loudly, never a silent partial ingest
Tamil-transliterated text (“thanni varala”)	Lexicon/fuzzy hit or triage — never a confident wrong domain
Resolve attempted with no docket	`GUARD_FAILED` 422
Resolve → reopen → resolve	Credit, reversal, new credit. Net +2, three ledger rows
Attender A acts on attender B’s task	403 `NOT_ASSIGNED`
Month boundary at 02:00 IST on the 1st	Falls in the new month
Anomaly detector below its n gate	Emits nothing
Push service returns `410 Gone`	Subscription revoked; no further attempts
Push unreachable for an hour	Retries, then `FAILED`. **No state transition affected**

9.3 Concurrency suite

Each test runs against real Postgres with threads released by a latch, and asserts on **final database state** rather than response codes.

	Setup	Assertion
X1	10 threads, one task, same idempotency key	Exactly 1 ledger row, +2 total, all 10 get 200
X2	10 threads, one task, *different* keys	Still exactly 1 ledger row
X3	Manager reassigns while attender resolves	One succeeds, loser gets 409, ledger matches the winner
X4	4 attenders complete a shared duty at once	4 ledger rows, task completed exactly once
X5	4 attenders claim a 1-slot duty at once	1 assignment, 3 receive `NO_SLOTS_LEFT`
X6	3 parallel ingestion runs over the same rows	No duplicate complaints
X9	**4 attenders × 25 tasks concurrently**	100 ledger rows, 200 points, zero lost updates, p95 $<$ 500 ms